

배치 발행 부분 실패 처리와 재처리 전략

ERC-1155 · BulkIssueService · 청크 분할 · PARTIAL_FAILURE · tokenId 비트 설계

ERC-1155

BulkIssueService

tokenId 비트 설계

ERC-1155 — 멀티 토큰 표준

1 / 3

ERC-20 · ERC-721 · ERC-1155 — 무엇이 다른가

	ERC-20	ERC-721	ERC-1155
토큰 단위	1종류, 대량 발행	tokenId마다 고유 소유자 1명, 수량 1개	tokenId별 종류 구분 여러 명이 같은 tokenId 보유 가능
대체 가능성	Fungible — 대체 가능	Non-Fungible	혼용 가능
수량(amount)	있음	없음 (항상 1)	있음
배치 전송	없음	없음	<code>safeBatchTransferFrom</code>
교보 사용	—	—	✓ 사용

교보생명 케이스: 이벤트 타입별 여러 종류의 NFT를 1개 컨트랙트로 관리. tokenId `1<<64` = 걷기 챌린지, `2<<64` = 건강검진, `3<<64` = 보험료 납입. 같은 이벤트의 NFT를 여러 명이 보유 — ERC-1155가 적합.

ERC-1155 — 멀티 토큰 표준

2 / 3

ERC-1155 핵심 함수 — 실제 표준 시그니처

단일 발행 · 배치 발행

```
// 단일 발행: 1명에게 1종류
function mint(
    address to,      // 수신자 1명
    uint256 id,      // 토큰 종류 ID
    uint256 amount,  // 수량 (교보: 항상 1)
    bytes data
) external;

// 배치 발행: 1명에게 여러 종류
function mintBatch(
    address to,      // 수신자 1명 ← 핵심!
    uint256[] ids,   // 토큰 종류 여러 개
    uint256[] amounts, // 각 종류별 수량
    bytes data
) external;
```

잔액 조회 · 전송

```
// 잔액 조회
function balanceOf(
    address account,
    uint256 id
) external view returns (uint256);

// 단일 전송
function safeTransferFrom(
    address from, address to,
    uint256 id,
    uint256 amount,
    bytes data
) external;

// 여러 종류 동시 전송
function safeBatchTransferFrom(...) external;
```

ERC-1155 설계 포인트: 1 컨트랙트가 여러 토큰 종류를 관리. 토큰 종류 추가 = 새 컨트랙트 배포 불필요.

ERC-1155 — 멀티 토큰 표준 3 / 3

mintBatch()의 흔한 오해 — 다중 수신자 발행이 아니다

❌ 오해

mintBatch()를 쓰면

15만 명에게 한 TX로 발행 가능

✓ 실제

mintBatch(address to, uint256[] ids, ...)

→ 수신자 1명에게 여러 종류 발행

다중 수신자 = 표준에 없음 → 커스텀 필요

왜 표준에 없는가: 500명 콜백 중 1명 revert 시

499명 성공분을 롤백할지 건너뛸지 — 표준이 결정 불가. 이 복잡성은
응용 레이어(VASP · BulkIssueService)에 위임.

다중 수신자 배치 → 커스텀 컨트랙트

```
// 표준에 없음 - 컨트랙트에 직접 구현
function mintToMany(
    address[] memory recipients,
    uint256 id,
    uint256 amount
) external onlyMinter {
    for (uint256 i = 0; i < recipients.length; i++) {
        _mint(recipients[i], id, amount, "");
    }
}
```

BulkIssueService는 컨트랙트를 직접 호출하지 않는다.

VASP API 호출만 담당 — 온체인 구현(커스텀 컨트랙트 여부)은 VASP
계약 스펙으로 확인.

왜 배치 발행인가

1 / 2

issueOne()을 15만 번 순차 호출하면

시나리오

"1월 걷기 챌린지 달성자" 집계: **15만 명**
이 15만 명에게 오늘 안에 NFT를 발행해야 한다.

S29 issueOne() 순차 호출 시

issueOne() 1건 $\approx$ 2초 (VASP 네트워크 지연 포함)
 $150,000\text{건} \times 2\text{초} = 300,000\text{초}$
 $\approx$ **3.5일** — 허용 불가

문제 1 — 처리 시간

3.5일 후에 NFT를 받는 사용자가 생긴다.
"오늘 달성 $\rightarrow$ 오늘 발행"이 전제인 이벤트 설계와 충돌.

문제 2 — 블록체인 TX 비용

$\text{issueOne()} \times 15\text{만} = \text{TX } 15\text{만 개.}$
각 TX마다 가스비 발생.
VASP가 커스텀 배치 컨트랙트를 지원하면
TX 수를 **대폭 줄일 수 있다.**

결론: 단건 발행 로직(issueOne)을 그대로 쓰는 건 규모 문제를 해결하지 못한다. 배치 전용 서비스가 필요하다.

왜 배치 발행인가

2 / 2

배치 발행이 필요한 두 가지 이유 · 청크 전략 개요

이유 ① 처리 시간 단축

15만명을 청크(500명 단위)로 분할.

청크 내 500건은 동시(병렬) 처리.

300청크 × 10초 = 약 50분

이유 ② 가스 효율

VASP가 커스텀 배치 컨트랙트 지원 시

청크당 1 TX → TX 수 1/500로 감소.

가스비 대폭 절감.

결과 비교

순차: 3.5일

배치: 약 50분

≈ 100배 단축

배치 처리 전략 구조

15만명 userIds[]

↓ _chunk(500)

청크 0 (500명)

청크 1 (500명)

청크 2 (500명)

... (총 300개)

청크 간: 순차 처리

Rate Limit 보호 + Nonce 보장

진행 상태 BulkJob에 기록

→

→

청크 내: 병렬 처리 (Promise.all)

실패 청크 chunkErrors 저장

retryFailedChunks() 재처리 가능

500건 동시 → 청크당 ≈ 10초

블록체인 Gas

왜 한 번에 무한히 많이 담을 수 없는가

Gas란?

블록체인의 모든 연산은 **Gas** 비용을 소비한다.

Gas = "블록체인 연산 단위"

현실 세계 비유

블록 하나 = **트럭 한 대** (적재 용량 한계 있음)

TX 하나 = **화물** (각각 다른 무게 = Gas 소비량)

Block Gas Limit = 트럭 최대 적재 무게

무게 초과 화물 → 트럭에 못 실음 → TX 거절

1,000건을 한 TX에 담으면:

$1,000 \times 50,000 \text{ gas} = 50,000,000 \text{ gas}$

Block Gas Limit (30,000,000) 초과

→ TX 자체 실패 → **1건도 발행 안 됨**

Gas ≠ 가스비(ETH)

Gas: 연산 복잡도의 단위 (고정된 기준).

가스비 = Gas × gasPrice (ETH 단위, 시장에서 결정).

Block Gas Limit은 블록에 담을 수 있는 총 Gas 상한선.

결론: 1 TX에 넣을 수 있는 mint 수에는 물리적 한계가 있다. 이 한계가 `CHUNK_SIZE = 500`의 근거다.

EVM Block Gas Limit

왜 `CHUNK_SIZE = 500`인가 — Gas 계산에서 도출된 상한선

항목	값
Block Gas Limit (최대)	~30,000,000 gas
NFT mint 1건 가스 (추정)	~50,000 gas
이론적 최대 mint 수	$30M \div 50k = 600$ 건
안전 버퍼 17% 적용	$600 \times 0.83 \approx 500$ 건

주의: 50,000 gas/건은 추정치.

컨트랙트 로직 복잡도, Solidity 버전, EVM 업그레이드에 따라 달라짐.
운영 배포 전 반드시 실제 가스 측정 재실행.

1,000건 한 TX에 넣으면

$$1,000 \times 50,000 = 50,000,000 \text{ gas}$$

Block gas limit 30,000,000 초과

→ TX 자체 실패

→ 1건도 발행 안 됨

500건 → 안전

$$500 \times 50,000 = 25,000,000 \text{ gas}$$

Block gas limit 내 포함

17% 버퍼 → 컨트랙트 로직 변화 흡수

→ 안정적 발행 보장

`CHUNK_SIZE = 500`은 규칙이 아니라 Gas 물리적 한계에서 역산한 상한값이다. 컨트랙트가 바뀌면 이 숫자도 바뀐다.

청크 분할 알고리즘

150,000명 → CHUNK_SIZE=500 → 300개 청크

```
_chunk<T>(
  arr: T[],
  size: number
): T[][] {
  const result: T[][] = [];
  for (let i = 0; i < arr.length; i += size) {
    result.push(arr.slice(i, i + size));
  }
  return result;
}
```

`slice(i, i+size)`는 배열 끝을 넘어도 에러 없이 있는 만큼만 반환. 마지막 청크가 500 미만이어도 자동 처리.

왜 public? 독립 단위 테스트 가능해야 한다. 경계값(1001명, 빈 배열, 딱 500명) 검증이 중요.

동작 예시 (size=3, arr=['u0'~'u9'])

i	slice(i, i+3)	청크
0	slice(0, 3)	['u0', 'u1', 'u2']
3	slice(3, 6)	['u3', 'u4', 'u5']
6	slice(6, 9)	['u6', 'u7', 'u8']
9	slice(9, 12)	['u9'] ← 1개만 (정상)

경계값 케이스

입력	결과
arr=[], size=500	[] (빈 배열)
500명, size=500	청크 1개 (500명)
501명, size=500	청크 2개 (500명 + 1명)

병렬 처리 vs 순차 처리 1 / 2

300개 청크를 Promise.all로 동시에 처리하면

병렬 처리 — Promise.all

```
// 모든 청크를 동시에 실행
await Promise.all(
  chunks.map(chunk => processChunk(chunk))
);
```

장점

빠르다.

300개 청크 동시 실행 시

총 시간 $\approx$ 청크 1개 처리 시간 (10초)

단점 — 3가지 문제

① Rate Limit 위반

초당 100 요청 허용인데 300개 동시 $\rightarrow$ HTTP 429 Too Many Requests

② Nonce 충돌

같은 지갑에서 여러 TX 동시 전송 $\rightarrow$ nonce 뒤엉킴 $\rightarrow$ TX 누락

③ 부분 실패 추적 어려움

어느 청크가 실패했는지 특정 어려움

결론: 청크 간 병렬 처리는 속도는 빠르지만 VASP Rate Limit과 Nonce 보장을 깨뜨린다.

병렬 처리 vs 순차 처리 2 / 2

순차 처리 선택 이유 · 청크 내부는 병렬 유지

순차 처리 — for...of

```
// 청크를 하나씩 순서대로 처리
for (const chunk of chunks) {
  await processChunk(chunk); // 완료 후 다음 청크
}
```

장점

Rate Limit 자동 준수

(청크 사이 처리 시간이 자연 간격)

Nonce 순서 보장

실패 청크 식별 용이

단점

$300\text{개} \times 10\text{초/청크} = 50\text{분}$

BulkIssueService 최종 선택

청크 간: 순차 — Rate Limit · Nonce 보장

청크 내 500건: 병렬 (Promise.all)

청크당 $\approx 10\text{초}$

결과: $300 \times 10\text{초} = 50\text{분}$

(3.5일 대비 약 100배 단축)

핵심 판단: 부분 실패 추적과 Rate Limit 안전성이 속도보다 중요하다.

부분 실패 허용 정책 1 / 2

청크 하나가 실패하면 — 전체 롤백 vs 부분 커밋

✗ 전체 롤백 정책

청크 0 성공 (500명 발행 완료)
청크 1 실패 (WalletNotFoundError)
→ 청크 0의 500명 NFT burn TX 발행
→ 처음부터 재실행

문제점:

성공한 500명까지 재처리 필요.
역등성 체크 없으면 이중 발행 위험.
15만 명 중 1명 실패로 전체 취소 → 비효율.

✓ 부분 커밋 정책

청크 0 성공 → 그대로 유지
청크 1 실패 → chunkErrors에 기록
→ 나중에 retryFailedChunks() 재처리
→ BulkJob: PARTIAL_FAILURE 상태

장점:

성공한 청크는 확정 — 재처리 불필요.
실패 원인 기록 → 선택적 재처리 가능.
운영자가 실패 청크만 확인 후 대응.

두 정책의 핵심 차이: 성공한 작업을 취소(롤백)하느냐 vs 실패한 작업만 기록하고 분리하느냐.

부분 실패 허용 정책

2 / 2

교보생명 케이스에서 부분 커밋이 맞는 세 가지 이유

① 사용자별 독립 권

사용자 A의 지갑이 미등록됐다고
사용자 B의 발행을 막을 이유가 없다.

보험 이벤트는 개인별 달성 조건을
각각 충족한 독립적 권리 발생이다.

② 재처리 가능

실패 청크는 `chunkErrors`에 기록된다.
`retryFailedChunks()`로 나중에 선택적 재처리.

지갑 미등록 → 사용자가 지갑 등록 후
해당 청크만 재실행하면 된다.

③ 운영 현실

15만 명 중 100명의 지갑 미등록으로
15만 건 전체를 취소하는 건 비효율.

실패 100명만 별도로 처리하는 게
운영 비용 · 리스크 모두 낮다.

PARTIAL_FAILURE → COMPLETED 경로: `retryFailedChunks()` 후 모든 실패 청크가 성공하면 COMPLETED로 전환. COMPLETED와 FAILED는 되돌릴 수 없다 — 재실행이 필요하면 새 BulkJob을 생성한다.

이 설계는 표준 ERC-1155가 다중 수신자 mint를 표준에 포함하지 않은 이유와 같다: 실패 정책은 응용 레이어가 결정해야 한다.

BulkJob — 상태 엔티티

배치 작업의 전체 이력을 한 객체에 담는다

```
type BulkJobStatus =  
  | 'RUNNING'  
  | 'COMPLETED'  
  | 'PARTIAL_FAILURE'  
  | 'FAILED';  
  
interface ChunkError {  
  chunkIndex: number;  
  requestIds: string[]; // 각 IssuanceRequest ID  
  status: 'success' | 'failed';  
  error?: string; // 실패 사유  
}
```

```
interface BulkJob {  
  id: string;  
  tokenId: bigint;  
  totalUsers: number;  
  totalChunks: number;  
  doneChunks: number; // 완료 청크 수  
  status: BulkJobStatus;  
  chunkErrors: ChunkError[]; // 성공+실패 전체  
  createdAt: Date;  
  updatedAt: Date;  
}
```

chunkErrors는 실패 청크만 담는 게 아니다.

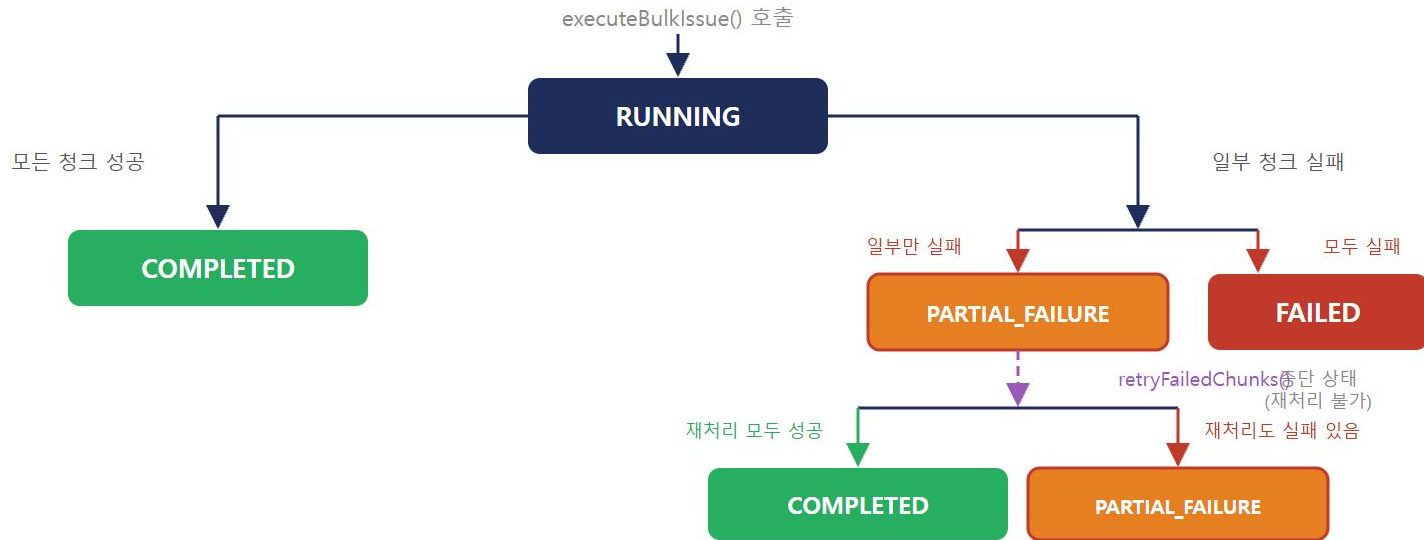
성공 청크도 status: 'success'로 기록 — 전체 청크 처리 이력을 한 곳에서 조회.

doneChunks: 진행률 계산에 사용.

$\text{progressPct} = \text{Math.round}(\text{doneChunks} / \text{totalChunks} \times 100)$
`retryFailedChunks()` 후 성공분도 `doneChunks`에 합산.

BulkJob 상태 전이

RUNNING · COMPLETED · PARTIAL_FAILURE · FAILED



MintSubmitter 인터페이스 1 / 3

IssuerService 연결 — 인터페이스 정의

```
interface MintSubmitter {  
  submitMintRequest(params: {  
    userId: string;  
    tokenId: bigint;  
    amount: bigint; // ← bigint (number 아님)  
  }): Promise<string>; // IssuanceRequest ID  
}
```

왜 bigint인가?

tokenId와 amount는 ERC-1155 uint256 타입.
JavaScript의 number는 53비트까지만 정확 표현 가능.
uint256 = 256비트 → bigint 필수.

DIP — Dependency Inversion Principle

BulkIssueService는 MintSubmitter

인터페이스에만 의존한다.

구체 구현(IssuerServiceSubmitter)을 모른다.

덕분에 테스트에서 S27~S29 전체 파이프라인
없이 S30 로직만 독립 검증 가능.

반환값 **string** = IssuanceRequest ID.

발행 결과 추적에 사용된다.

MintSubmitter 인터페이스

2 / 3

의존성 연결 구조 — S27~S30 전체 파이프라인

BulkIssueService

└ BulkJobRepository

└ └ PostgresBulkJobRepository ← 운영

└ └ InMemoryBulkJobRepository ← 실습

└ MintSubmitter

└ └ IssuerServiceSubmitter ← 운영

└ └ └ IssuerService (S29)

└ └ └ └ IssuancePolicyService (S28)

└ └ └ └ EventConditionService (S28)

└ └ └ └ WalletProvisioningService (S27)

└ └ └ └ IssuanceRequestRepository

└ └ └ └ VaspClient

└ StubMintSubmitter ← 테스트

레이어 분리의 의미

BulkIssueService

"어떻게 배치 처리할 것인가"

IssuerService (S29)

"단건 발행 정책 적용"

MintSubmitter 인터페이스

두 레이어를 느슨하게 연결

테스트: Stub 주입 → S27~S29 전체 불필요.

운영: IssuerServiceSubmitter 주입 → 전체 파이프라인 자동 연결.

MintSubmitter 인터페이스

3 / 3

IssuerServiceSubmitter — Adapter 패턴

```
class IssuerServiceSubmitter
  implements MintSubmitter {

  constructor(
    private readonly issuerSvc: IssuerService
  ) {}

  async submitMintRequest(params: {
    userId: string; tokenId: bigint; amount: bigint;
  }): Promise<string> {
    const { userId, tokenId } = params;
    const result = await this.issuerSvc.issueOne({
      userId,
      eventType: tokenIdToEventType(tokenId), // §8-1
      data: {},
    });
    if (!result.eligible || !result.requestId)
      throw new Error('발행 조건 미충족');
    return result.requestId;
  }
}
```

```
// ㉠ 운영 환경 조립 (NestJS)
@Module({
  imports: [IssuerModule, BulkJobRepoModule],
  providers: [
    {
      provide: 'MintSubmitter',
      useClass: IssuerServiceSubmitter,
    },
    BulkIssueService,
  ],
})
export class BulkIssueModule {}

// ㉡ 테스트 환경 - Stub 주입
const stubSubmitter: MintSubmitter = {
  async submitMintRequest({ userId }) {
    return `stub-req-${userId}`;
  },
};

const svc = new BulkIssueService(
  new InMemoryBulkJobRepository(),
  stubSubmitter,
);
```

getJobStatus()

1 / 2

progress와 progressPct 계산 — 단순 레코드 반환이 아니다

```
async getJobStatus(jobId: string)
: Promise<BulkJob & {
  progress: string;
  progressPct: number;
}> {

  const job = await this.jobRepo.findById(jobId);
  if (!job) throw new Error(`Job not found: ${jobId}`);

  return {
    ...job,
    progress: `${job.doneChunks}/${job.totalChunks}`,
    progressPct: job.totalChunks > 0
      ? Math.round(
          (job.doneChunks / job.totalChunks) * 100
        )
      : 0,
  };
}
```

totalChunks > 0 가드가 필요한 이유:

빈 `userIds[]`로 `executeBulkIssue()` 호출 시

$\text{totalChunks} = 0 \rightarrow 0 / 0 = \text{NaN}$

→ 가드 없으면 `progressPct: NaN` 반환

progress vs progressPct

progress: "1/2" — 운영자용 가독 문자열

progressPct: 50 — UI 프로그레스바용 숫자

DB에 저장하지 않고 **조회 시 계산**. `doneChunks · totalChunks`가 source of truth.

getJobStatus()

2 / 2

운영자 폴링 응답 예시 — GET /admin/bulk-jobs/{jobId}

PARTIAL_FAILURE 상태 응답

```
{
  "id": "bulk-job-uuid-001",
  "status": "PARTIAL_FAILURE",
  "totalUsers": 1000,
  "totalChunks": 2,
  "doneChunks": 1,
  "progress": "1/2",
  "progressPct": 50,
  "chunkErrors": [
    {
      "chunkIndex": 0,
      "status": "success",
      "requestIds": ["req-0", "..."]
    },
    {
      "chunkIndex": 1,
      "status": "failed",
      "error": "WalletNotFoundError: user-500"
    }
  ]
}
```

폴링 흐름

executeBulkIssue() → jobId 즉시 반환

운영자/시스템이 주기적으로

GET /admin/bulk-jobs/{jobId} 호출

RUNNING → progressPct 증가 확인

COMPLETED → 완료

PARTIAL_FAILURE → 실패 체크 확인 후 retry

Phase 1: 폴링 (GET 반복 호출)

Phase 2: SSE(Server-Sent Events)로 전환 — 서버가 진행 상황을 실시간 Push

chunkErrors[0].status = 'success'도 포함 — 전체 체크 이력 한 번에 조회 가능.

Promise.all vs Promise.allSettled

1 / 2

현재 구현 — Promise.all의 동작과 한계

현재 구현 (Phase 1)

```
// 청크 내 500건 동시 처리
const requestIds = await Promise.all(
  chunk.map(userId =>
    this.submitter.submitMintRequest({
      userId, tokenId, amount
    })
  )
);
```

Promise.all의 동작:

499건 성공 + 1건 실패

→ 전체 reject — 청크 전체 실패로 기록.

499건 성공 여부를 알 수 없음.

왜 Phase 1에서 Promise.all을 쓰는가

구현 단순성:

성공 = requestIds 배열 그대로 사용.

실패 = catch로 청크 전체를 chunkErrors에 기록.

500건 중 1건 실패 시 청크 전체 retry

→ 멍등성(IssuanceRequest 중복 방지)이 보호.

재처리 시 499건은 idempotency guard로 skip.

Phase 1 트레이드오프:

구현 단순 ↔ 청크 내 개별 실패 추적 불가.

운영 규모가 커지면 Phase 2로 전환 필요.

Promise.all vs Promise.allSettled

2 / 2

Phase 2 — Promise.allSettled로 개별 실패 추적

Phase 2 구현

```
const results = await Promise.allSettled(
  chunk.map(userId =>
    this.submitter.submitMintRequest({
      userId, tokenId, amount
    })
  )
);

// 성공 건만 추출
const successIds = results
  .filter((r): r is PromiseFulfilledResult<string>
    => r.status === 'fulfilled')
  .map(r => r.value);

// 실패 userId만 추출
const failedUserIds = chunk
  .filter((_, i) =>
    results[i].status === 'rejected'
  );
```

	Promise.all	Promise.allSettled
동작	1건 실패 → 즉시 전체 reject	모두 끝날 때까지 기다림
결과 형태	성공 값 배열 or Error	{ status, value/reason }[]
개별 추적	불가	가능
구현 복잡도	낮음	높음
Phase	Phase 1	Phase 2

Phase 2에서는 500건 중 1건 실패해도 499건 성공 처리.
실패한 userId만 별도 chunkErrors에 기록 → 더 세밀한 재처리.

tokenId 비트 설계

1 / 3

uint256 = 256비트 — 하나의 숫자에 여러 정보를 담는다

uint256 = 256비트 — 현재 설계



향후 확장 레이아웃 예시



현재 설계: 상위 64비트에 이벤트 타입 코드 저장.

$\text{BigInt}(1) \ll \text{BigInt}(64) = \text{WALK_CHALLENGE tokenId}$

$\text{BigInt}(2) \ll \text{BigInt}(64) = \text{HEALTH_CHECK tokenId}$

$\text{BigInt}(3) \ll \text{BigInt}(64) = \text{PREMIUM_PAYMENT tokenId}$

왜 단순 순번(1, 2, 3)이 아닌가?

이벤트 타입 vs 인스턴스 ID 구분 불가.

나중에 필드 추가 시 기존 ID 충돌 발생.

비트 분할 → 필드 확장을 설계에 내장.

tokenId 비트 설계

2 / 3

tokenId 생성 · tokenIdToEventType() 역변환

tokenId 생성 — 비트 시프트

```
// 이벤트 코드를 상위 64비트 위치로 밀어 올림
const WALK      = BigInt(1) << BigInt(64);
// = 18446744073709551616n (= 2^64)

const HEALTH    = BigInt(2) << BigInt(64);
// = 36893488147419103232n (= 2 × 2^64)

const PREMIUM   = BigInt(3) << BigInt(64);
// = 55340232221128654848n (= 3 × 2^64)
```

비트 시프트 의미:

$\text{BigInt}(1) \ll \text{BigInt}(64)$
= ...00000001 [64개의 0]
오른쪽으로 64번 밀기($\gg 64$) → 1 복원

tokenIdToEventType() — 역변환

```
function tokenIdToEventType(tokenId: bigint): string {
  const MAP: Record<string, string> = {
    '1': 'WALK_CHALLENGE',
    '2': 'HEALTH_CHECK',
    '3': 'PREMIUM_PAYMENT',
  };
  // 상위 64비트 추출: 오른쪽으로 64비트 시프트
  const code = String(tokenId >> BigInt(64));
  const type = MAP[code];
  if (!type) throw new Error(`알 수 없는 tokenId: ${tokenId}`);
  return type;
}

// 사용 예
tokenIdToEventType(WALK);    // 'WALK_CHALLENGE'
tokenIdToEventType(HEALTH);  // 'HEALTH_CHECK'
```

IssuerServiceSubmitter 내부에서 사용.
tokenId(bigint) → eventType(string) 변환 후
issuerSvc.issueOne({ eventType }) 호출.

tokenId 비트 설계

3 / 3

향후 확장 — 남은 192비트로 무엇을 더 만들 수 있는가

① 발행 연도 — 빈티지 NFT

2026년 NFT vs 2027년 NFT → tokenId 다름.

"2026 오리지널 참여자" 진위 검증.

초기 참여자에게 희소 토큰 부여.

② 캠페인 코드 — 온체인 기록

동일 경기 챌린지라도 어느 캠페인인지

DB 조회 없이 tokenId 하나로 식별.

Etherscan에서 즉시 읽힘.

③ 개인별 시리얼 번호

하위 128비트에 수령자 순번 저장.

"당신은 1번 참여자입니다" 온체인 증명.

ERC-1155 위의 진짜 "1 of 1" 구현.

④ 등급(Tier) 인코딩

Bronze · Silver · Gold → tokenId에 내장.

컨트랙트가 tokenId만 보고 혜택 자동 분기.

할인율, 서비스 접근권 등 차별화.

온체인 로직의 자율성

일반 설계:

tokenId → 백엔드 API → DB 조회 → 결과

비트 설계:

tokenId >> 64 → 이벤트 타임 즉시 추출

서드파티 DeFi · 다른 컨트랙트 · 지갑 앱

어떤 주체도 **교보생명 서버에 의존하지 않고**

tokenId 하나로 토큰의 모든 의미를 읽는다.

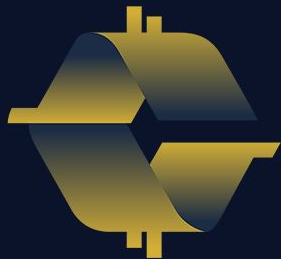
이것이 온체인 데이터의 진짜 가치다:

토큰이 자기 자신의 의미를 설명한다 (Self-describing).

실습

S30 실습 — 7개 섹션

- [1] `_chunk()` 알고리즘 — 청크 분할 (경계값 포함)
- [2] 전체 성공 → COMPLETED
- [3] 부분 실패 → PARTIAL_FAILURE (청크 1 실패)
- [4] 전체 실패 → FAILED (alwaysFail)
- [5] `retryFailedChunks` → COMPLETED 전환
- [6] `Promise.all` — 청크 내 1건 실패 → 청크 전체 실패
- [7] `getJobStatus` `progressPct` 계산



COINCRAFT
ACADEMY

[1] _chunk() 알고리즘 — 배열 청크 분할

npm run exercise:s30:1 · CHUNK_SIZE=500 · 마지막 청크 자동 처리

```
$ npm run exercise:s30:1
```

실행 코드

```
const svc = new BulkIssueService(  
  new InMemoryBulkJobRepository(),  
  makeSubmitter()  
);  
  
const arr1000 = Array.from(  
  { length: 1000 }, (_, i) => `user-${i}`);  
const arr1001 = Array.from(  
  { length: 1001 }, (_, i) => `user-${i}`);  
  
const c1000 = svc._chunk(arr1000, 500);  
const c1001 = svc._chunk(arr1001, 500);  
const c0 = svc._chunk([], 500);  
  
console.log(`1000명 → ${c1000.length}청크 [${c1000.map(c => c.length).join()}]`);  
console.log(`1001명 → ${c1001.length}청크 [${c1001.map(c => c.length).join()}]`);
```

핵심 포인트

slice(i, i+size) 자동 처리:

배열 끝 초과 시 있는 만큼만 반환

1001명 → 마지막 청크 1명

별도 나머지 처리 코드 불필요

CHUNK_SIZE = 500 (Block Gas Limit 기준):

ERC-1155 safeBatchTransferFrom 1트랜잭션 제한

가스 초과 → 트랜잭션 실패

구현:

```
for (let i = 0; i < arr.length; i += size)  
  result.push(arr.slice(i, i + size));
```

[2] 전체 성공 → COMPLETED

npm run exercise:s30:2 · 10명 → 1청크 → 전체 성공 → COMPLETED

```
$ npm run exercise:s30:2
```

실행 코드

```
const repo = new InMemoryBulkJobRepository();
const svc = new BulkIssueService(
  repo, makeSubmitter());
const users = Array.from(
  { length: 10 }, (_, i) => `user-${i}`);

const jobId = await svc.executeBulkIssue({
  userIds: users, tokenId: TOKEN_ID
});
const job = await svc.getJobStatus(jobId);

console.log(`status: ${job.status}`);
console.log(`progress: ${job.progress}`);
console.log(`progressPct: ${job.progressPct}%`);
```

핵심 포인트

최종 상태 결정 로직:

failedCount === 0 → COMPLETED

failedCount === totalChunks → FAILED

그 외 → PARTIAL_FAILURE

10명 < 500 → 1청크:

doneChunks=1, totalChunks=1

progress="1/1", progressPct=100

jobId 즉시 반환:

executeBulkIssue()는 jobId 즉시 반환

폴링: GET /bulk-jobs/:id/status

[3] 부분 실패 → PARTIAL_FAILURE

npm run exercise:s30:3 · 1000명 · 청크 1 실패 → PARTIAL_FAILURE

```
$ npm run exercise:s30:3
```

실행 코드

```
const svc = new BulkIssueService(
  repo,
  makeSubmitter({ failForUsersFrom: 500 })
);
// user-500 이상 실패
const users = Array.from(
  { length: 1000 }, (_, i) => `user-${i}`);

const jobId = await svc.executeBulkIssue({
  userIds: users, tokenId: TOKEN_ID
});
const job = await svc.getJobStatus(jobId);
const failed = job.chunkResults
  .filter(c => c.status === 'failed');
console.log(`실패 청크 인덱스: ${failed.map(c => c.chunkIndex)}`);
```

핵심 포인트

순차 처리 보장:

for loop → 청크 간 순차 실행
청크 0 실패해도 청크 1 계속 처리
이미 완료된 청크는 재처리 안 함

chunkResults 기록:

성공: { status: 'success', requestIds }
실패: { status: 'failed', error }
retryFailedChunks()가 이 인덱스로 재처리

PARTIAL_FAILURE의 의미:

일부 사용자는 NFT 발행됨 / 일부는 미발행
운영자가 실패 청크만 선택 재처리 가능

[4] 전체 실패 → FAILED

npm run exercise:s30:4 · alwaysFail → 모든 청크 실패 → FAILED

```
$ npm run exercise:s30:4
```

실행 코드

```
const svc = new BulkIssueService(
  repo,
  makeSubmitter({ alwaysFail: true })
);
const users = Array.from(
  { length: 20 }, (_, i) => `user-${i}`);

const jobId = await svc.executeBulkIssue({
  userIds: users, tokenId: TOKEN_ID
});
const job = await svc.getJobStatus(jobId);

console.log(`status: ${job.status}`);
console.log(`doneChunks: ${job.doneChunks}`);
```

핵심 포인트

failedCount === totalChunks → FAILED:

성공 청크 0개 → 전체 실패

doneChunks=0 : 발행 성공한 사용자 없음

FAILED vs PARTIAL_FAILURE:

FAILED: 전체 재처리 필요

PARTIAL_FAILURE: 실패 청크만 재처리

→ 운영자가 상태로 재처리 범위 판단

중단 없는 순차 처리:

청크 실패 시 다음 청크 계속 진행

try/catch: 에러 기록 후 continue

모든 청크 처리 후 최종 상태 결정

[5] retryFailedChunks → COMPLETED 전환

npm run exercise:s30:5 · PARTIAL_FAILURE → 실패 청크만 재처리 → COMPLETED

```
$ npm run exercise:s30:5
```

실행 코드

```
const users = Array.from(
  { length: 1000 }, (_, i) => `user-${i}`);

// 1차: 청크 1 실패 → PARTIAL_FAILURE
const svc1 = new BulkIssueService(
  repo, makeSubmitter({ failForUsersFrom: 500 }));
const jobId = await svc1.executeBulkIssue({
  userIds: users, tokenId: TOKEN_ID
});
const before = await svc1.getJobStatus(jobId);
console.log(`재처리 전: ${before.status}`);

// 2차: 실패 청크만 재처리 (이번엔 성공)
const svc2 = new BulkIssueService(
  repo, makeSubmitter()); // 전부 성공
await svc2.retryFailedChunks(jobId, users);
const after = await svc2.getJobStatus(jobId);
console.log(`재처리 후: ${after.status}`);
```

핵심 포인트

실패 청크 인덱스만 재처리:

chunkResults에서 failed만 필터링

성공 청크는 재처리 안 함 — 이중 발행 방지

in-place 업데이트:

failed 항목을 success로 덮어쓰기

모든 청크 success → COMPLETED로 전환

서로 다른 submitter 사용 가능:

1차: failForUsersFrom=500 submitter

2차: 정상 submitter (근본 원인 해결 후)

동일 jobRepo 공유 → 이전 상태 유지

[6] Promise.all — 청크 내 1건 실패 → 청크 전체 실패

npm run exercise:s30:6 · user-3만 실패 → 청크 전체 reject

```
$ npm run exercise:s30:6
```

실행 코드

```
// user-3만 실패하는 submitter
const submitter: MintSubmitter = {
  async submitMintRequest({ userId }) {
    console.log(`[MINT] ${userId}`);
    if (userId === 'user-3')
      throw new Error(
        `WalletNotFoundError: ${userId}`);
    return `req-${userId}`;
  },
};

const users = ['user-0', 'user-1', 'user-2',
  'user-3', 'user-4'];

const svc = new BulkIssueService(repo, submitter);
const jobId = await svc.executeBulkIssue({
  userIds: users, tokenId: TOKEN_ID
});

const job = await svc.getJobStatus(jobId);
```

핵심 포인트

Promise.all: 1건 실패 → 즉시 reject:

나머지 4건 성공해도 청크 전체 실패
현재 구현의 트레이드오프

청크 내 병렬 실행:

user-0~4 동시 submitMintRequest 호출
1건 reject → Promise.all reject → catch

Phase 2: Promise.allSettled 개선:

개별 실패 허용 → 성공/실패 분리 기록
청크 내 1명 실패 → 청크 전체 재처리 불필요
현재는 트레이드오프 인지하고 설계

[7] getJobStatus progressPct 계산

npm run exercise:s30:7 · 600명 → 2청크 · $\text{Math.round}(\text{doneChunks}/\text{totalChunks}*100)$

```
$ npm run exercise:s30:7
```

실행 코드

```
const users = Array.from(
  { length: 600 }, (_, i) => `user-${i}`);
const svc = new BulkIssueService(
  repo, makeSubmitter());

const jobId = await svc.executeBulkIssue({
  userIds: users, tokenId: TOKEN_ID
});
const job = await svc.getJobStatus(jobId);

console.log(`totalUsers: ${job.totalUsers}`);
console.log(`totalChunks: ${job.totalChunks}`);
console.log(`progress: ${job.progress}`);
console.log(`progressPct: ${job.progressPct}%`);
```

핵심 포인트

600명 → 2청크 (500 + 100):

totalChunks=2, doneChunks=2 (전부 성공)

progressPct = $\text{Math.round}(2/2*100)$ = 100

progress 문자열 vs progressPct 숫자:

progress = "2/2" — 운영자 대시보드 표시

progressPct = 100 — 프로그레스바 렌더링

totalChunks=0 방어:

$\text{totalChunks} > 0 ? \text{Math.round}(...) : 0$

빈 배열 엡지 케이스 — 0으로 처리

운영자 폴링: GET /bulk-jobs/:id/status

S30 · BULKISSUESERVICE — ERC-1155 배치 발행 구현

Thank You

BulkIssueService · _chunk() · executeBulkIssue()
PARTIAL_FAILURE · retryFailedChunks() · Promise.all · progressPct

M5 · S30

실습 완료

kyobo-digital-asset-platform

COINCRAFT
ACADEMY